

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Лабораторная работа №3

Решение систем нелинейных уравнений

(по дисциплине «Вычислительная математика»)

Выполнил:

Лабин Макар Андреевич,
ВЫЧМ 3.2, Р3231, 466449.

Проверила:

Перл Ольга Вячеславовна



г. Санкт-Петербург, Россия
2026

Содержание

1	Постановка задач	1
2	Выполнение работы	2
2.1	Описание метода	2
2.1.1	Начальные данные	2
2.1.2	Условия сходимости	2
2.1.3	Критерий останова и его обоснование	3
2.2	Блок-схема численного метода	3
2.3	Листинг численного метода	4
2.4	Результаты работы программы	8
3	Заключение	10

1 Постановка задач

Дана система нелинейных уравнений вида $F(X) = 0$. По заданному начальному приближению $X^{(0)}$ необходимо найти решение системы с точностью до 5-го верного знака после запятой.

В варианте №2 решение осуществляется *методом простых итераций*.

То есть в ходе лабораторной работы будут выполнены следующие задачи:

1. Приведение исходной системы нелинейных уравнений к итерационному виду $X = \Phi(X)$, который подходит для применения метода.
2. Разработка алгоритма, который реализует метод простых итераций для системы уравнений.
3. Вычисление вектора корней системы по заданным начальным приближениям и выбранному номеру системы.

2 Выполнение работы

2.1 Описание метода

2.1.1 Начальные данные

Рассмотрим систему нелинейных уравнений размерности n в векторном виде:

$$\vec{F}(\vec{x}) = \vec{0}, \quad \vec{F}(\vec{x}) = (f_1(x_1, \dots, x_n), \dots, f_n(x_1, \dots, x_n))^T.$$

Для применения метода простых итераций исходную систему необходимо преобразовать к эквивалентному виду

$$\vec{x} = \Phi(\vec{x}),$$

где $\Phi : \mathbb{R}^n \rightarrow \mathbb{R}^n$ — некоторая вектор-функция. Такое преобразование не является единственным. Сходимость метода напрямую зависит от выбора Φ .

2.1.2 Условия сходимости

Для сходимости итерационного процесса $\vec{x}^{(k+1)} = \Phi(\vec{x}^{(k)})$ исследуем невязку:

$$\vec{e}^{(k)} = \vec{x}^{(k)} - \vec{x}^*$$

Рассмотрим класс функций, дифференцируемых в точке $\vec{x}^*$. Разложим $\Phi(\vec{x})$ в ряд Тейлора в окрестности $\vec{x}^*$:

$$\Phi(\vec{x}^{(k)}) = \Phi(\vec{x}^*) + J_\Phi(\vec{x}^*)(\vec{x}^{(k)} - \vec{x}^*) + O(\|\vec{x}^{(k)} - \vec{x}^*\|^2)$$

где $J_\Phi(\vec{x}^*)$ — матрица Якоби. Учтём, что $\vec{x}^{(k+1)} = \Phi(\vec{x}^{(k)})$ и $\vec{x}^* = \Phi(\vec{x}^*)$:

$$\begin{aligned} \vec{x}^{(k+1)} &\approx \vec{x}^* + J_\Phi(\vec{x}^*)(\vec{x}^{(k)} - \vec{x}^*) \\ \vec{e}^{(k+1)} &\approx J_\Phi(\vec{x}^*)\vec{e}^{(k)} \end{aligned}$$

То есть условие сходимости сводится к требованию, чтобы спектральный радиус матрицы Якоби в точке решения был меньше единицы:

$$q \equiv \rho(J_\Phi(\vec{x}^*)) < 1$$

Из этого можем вывести следствие: если в некоторой окрестности решения существует такая согласованная матричная норма, что:

$$\|J_\Phi(\vec{x})\| \leq q < 1,$$

то итерационная последовательность сходится к решению $\vec{x}^*$ *линейно*:

$$\|\vec{e}^{(k+1)}\| \leq q \|\vec{e}^{(k)}\|$$

2.1.3 Критерий останова и его обоснование

Для контроля точности итерационного процесса необходимо установить связь между расстоянием до точного решения $\|\vec{x}^{(k)} - \vec{x}^*\|$ и разностью двух последовательных приближений $\|\vec{x}^{(k)} - \vec{x}^{(k-1)}\|$.

Оценим расстояние от текущей итерации до корня как сумму всех последующих шагов через неравенство треугольника:

$$\|\vec{x}^{(k)} - \vec{x}^*\| = \left\| \sum_{i=k}^{\infty} (\vec{x}^{(i)} - \vec{x}^{(i+1)}) \right\| \leq \sum_{i=k}^{\infty} \|\vec{x}^{(i)} - \vec{x}^{(i+1)}\|$$

Учтём, что каждый шаг уменьшается пропорционально q :

$$\|\vec{x}^{(i)} - \vec{x}^{(i+1)}\| \leq q^{i-k} \|\vec{x}^{(k)} - \vec{x}^{(k+1)}\|$$

Подставляя это в сумму, получим убывающую геометрическую прогрессию:

$$\|\vec{x}^{(k)} - \vec{x}^*\| \leq \|\vec{x}^{(k)} - \vec{x}^{(k+1)}\| \cdot (1 + q + q^2 + \dots) = \frac{1}{1-q} \|\vec{x}^{(k)} - \vec{x}^{(k+1)}\|$$

Воспользуемся поправкой на один шаг и получим финальную оценку:

$$\|\vec{x}^{(k)} - \vec{x}^*\| \leq \frac{q}{1-q} \|\vec{x}^{(k)} - \vec{x}^{(k-1)}\|$$

Таким образом, для достижения заданной точности ε расчёт необходимо вести до выполнения условия:

$$\|\vec{x}^{(k)} - \vec{x}^*\| \leq \varepsilon \implies \|\vec{x}^{(k)} - \vec{x}^{(k-1)}\| \leq \frac{1-q}{q} \varepsilon$$

Если метод сходится быстро, то коэффициент перед погрешностью в финальной оценке становится меньше или равен единице:

$$q \leq 0.5 \implies \frac{q}{1-q} \leq 1.$$

В этом случае условие $\|\vec{x}^{(k)} - \vec{x}^{(k-1)}\| \leq \varepsilon$ гарантирует, что истинная погрешность $\|\vec{x}^{(k)} - \vec{x}^*\|$ также не превышает ε .

2.2 Блок-схема численного метода

На вход реализуемого метода подаются следующие данные:

- $\Phi(x)$ — функция, реализующая эквивалентную систему уравнений вида $x = \Phi(x)$.
- x_{init} — вектор начального приближения размерности n ; состоит из вещественных чисел двойной точности.
- ε — требуемая точность вычислений; вещественное число двойной точности.
- M — максимально допустимое количество итераций; целое число.

На выходе метода ожидается вектор над вещественными числами двойной точности — найденное решение системы, а также целое число — количество затраченных итераций.

Остановка итерационного процесса происходит либо при достижении заданной точности ε , либо при превышении лимита итераций M .

Далее приводится блок-схема, реализующая данный численный метод.

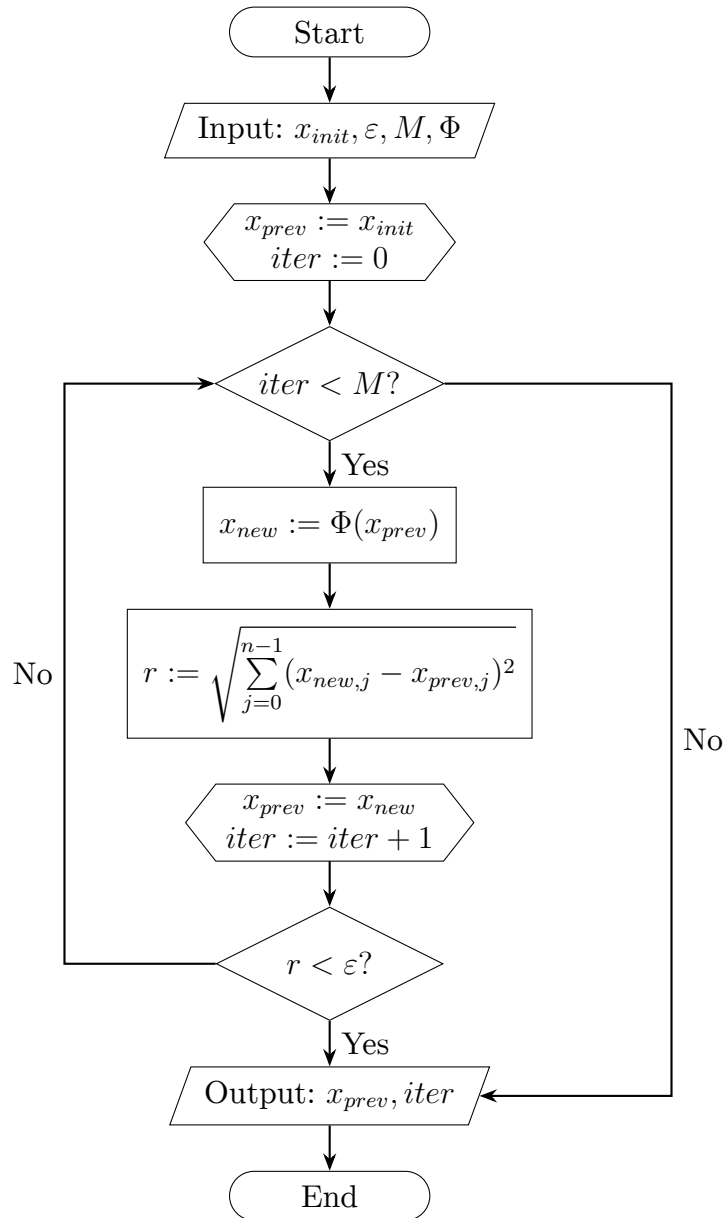


Схема 1 — Блок-схема метода простых итераций (с метаданными).

2.3 Листинг численного метода

Для реализации блок-схем воспользуемся языком программирования C++.

```

1 #ifndef ITERATIONS_H
2 #define ITERATIONS_H
3
4 #include "preamble.h"
  
```

```

5
6 #include <unordered_map>
7 #include <string>
8
9 struct ResultInfo {
10     std::vector<double> solution;
11     std::size_t iterations_count;
12 };
13
14 class SimpleIterationsSystem {
15     using SystemHandler = std::function<std::vector<double>(const
        std::vector<double>& x)>;
16
17 public:
18     SimpleIterationsSystem(const double eps, const std::size_t
        max_iterations)
19         : _eps(eps), _max_iterations(max_iterations), _handlers
            ({} ) {
20     }
21
22     const SystemHandler& GetHandler(std::size_t system_num) {
23         if (this->_handlers.count(system_num) != 0U) {
24             return this->_handlers.at(system_num);
25         }
26         throw std::invalid_argument("no handlers for specified
            system: " + std::to_string(system_num));
27     }
28
29     void SetHandler(std::size_t system_num, const SystemHandler&
        handler) {
30         this->_handlers[system_num] = handler;
31     }
32
33     const ResultInfo SolveSystem(std::size_t system_num, const std
        ::vector<double>& init_x) {
34         std::vector<double> prev_x(init_x);
35
36         std::size_t iterations_count = 0;
37         for (std::size_t i = 0; i < this->_max_iterations; i++) {
38             const SystemHandler& handler = GetHandler(system_num);
39             const std::vector<double> new_x = handler(prev_x);
40
41             double residual = 0.0;
42             for (std::size_t j = 0; j < new_x.size(); j++) {
43                 residual += std::pow(new_x[j] - prev_x[j], 2);
44             }
45             residual = std::sqrt(residual);
46
47             prev_x = new_x;
48             iterations_count++;
49             if (residual < this->_eps) {

```

```

50         break;
51     }
52 }
53 return { prev_x, iterations_count };
54 }
55
56 private:
57     const double _eps;
58     const std::size_t _max_iterations;
59     std::unordered_map<std::size_t, SystemHandler> _handlers;
60 };
61
62 const SimpleIterationsSystem InitSystem() {
63     SimpleIterationsSystem obj = SimpleIterationsSystem(1e-7, 1e5)
64     ;
65     obj.SetHandler(0, [](const std::vector<double>& x) {
66         return x;
67     });
68     obj.SetHandler(1, [](const std::vector<double>& x) {
69         std::vector<double> new_x(x.size());
70
71         new_x[0] = x[0] - sin(x[0]);
72         new_x[1] = x[1] - (x[0] * x[1] / 2.0);
73
74         return new_x;
75     });
76     obj.SetHandler(2, [](const std::vector<double>& x) {
77         std::vector<double> new_x(x.size());
78
79         new_x[0] = -pow(9.0 * x[1] - 2.0, 0.25);
80         new_x[1] = cbrt((x[0] * x[0] * x[1] * x[1] - 3.0 * pow(x[0],
81             3.0) + 8.0) / 6.0);
82
83         return new_x;
84     });
85     obj.SetHandler(3, [](const std::vector<double>& x) {
86         std::vector<double> new_x(x.size());
87
88         new_x[0] = 0.1 - pow(x[0], 2.0) + 2.0 * x[1] * x[2];
89         new_x[1] = -0.2 - pow(x[1], 2.0) - 3.0 * x[0] * x[2];
90         new_x[2] = 0.3 - pow(x[2], 2.0) - 2.0 * x[0] * x[1];
91
92         return new_x;
93     });
94     obj.SetHandler(4, [](const std::vector<double>& x) {
95         std::vector<double> new_x(x.size());
96
97         new_x[0] = cbrt(x[1]);
98         new_x[1] = sqrt(1.0 - x[0] * x[0]);
99
100        return new_x;

```

```

99     });
100     obj.SetHandler(5, [](const std::vector<double>& x) {
101         std::vector<double> new_x(x.size());
102
103         new_x[0] = log(2.0 - x[0]);
104         new_x[1] = x[0] - sin(x[1]);
105
106         return new_x;
107     });
108     return obj;
109 }
110
111 namespace {
112 vector<double> solve_by_fixed_point_iterations(
113     int system_id, int number_of_unknowns, vector<double> x
114 ) {
115     SimpleIterationsSystem obj = InitSystem();
116     return obj.SolveSystem(system_id, x).solution;
117 }
118
119 std::vector<double> CaluclateResidues(const std::size_t
120     system_num, const std::vector<double>& result) {
121     funcvec_t funcs = get_functions(system_num);
122     std::vector<double> residues(funcs.size());
123     for (std::size_t i = 0; i < funcs.size(); i++) {
124         residues[i] = -funcs[i](result);
125     }
126     return residues;
127 }
128 } // namespace
129 #endif

```

ЛИСТИНГ 1 — Файл iterations.hpp

```

1  #include "lib/iterations.hpp"
2
3  #include <iostream>
4  #include <unordered_map>
5
6  int main() {
7      SimpleIterationsSystem obj = InitSystem();
8
9      std::unordered_map<std::size_t, std::vector<double>> initial_x
10         = {
11         {1, { 0.0, 0.0 }},
12         {2, { -1.9, 2.1 }},
13         {3, { 0.0, 0.0, 0.0 }},
14         {4, { 0.8, 0.6 }},
15         {5, { 0.4, 0.2 }},
16     };
17     for (std::size_t i = 1; i <= 5; i++) {

```

```

17     std::cout << "--- Solution for system #" << i << ":\n";
18     ResultInfo rawInfo = obj.SolveSystem(i, initial_x[i]);
19     std::vector<double> result = rawInfo.solution;
20     std::vector<double> residues = CaluclateResidues(i, result);
21     for (std::size_t j = 0; j < result.size(); j++) {
22         std::cout << "x[" << j << "]: " << result[j] << "\t\t" <<
23             "res[" << j << "]: " << residues[j] << "\n";
24     }
25     double mse = 0.0;
26     for (const auto& res : residues) {
27         mse += res * res;
28     }
29     mse /= residues.size();
30     std::cout << "MSE: " << mse << "\n";
31     std::cout << "Total iterations: " << rawInfo.
32         iterations_count << "\n";
33 }
34 }

```

Листинг 2 — Файл main.cpp

2.4 Результаты работы программы

Для демонстрации работы метода и тестирования программы рассмотрим пять систем различной природы.

1. *Тригонометрическая система:*

$$\begin{cases} \sin(x_1) = 0 \\ \frac{x_1 x_2}{2} = 0 \end{cases} \implies \begin{cases} x_1 = x_1 - \sin(x_1) \\ x_2 = x_2 - \frac{x_1 x_2}{2} \end{cases}$$

2. *Алгебраическая система высоких степеней:*

$$\begin{cases} x_1^2 x_2^2 - 3x_1^3 - 6x_2^3 + 8 = 0 \\ x_1^4 - 9x_2 + 2 = 0 \end{cases} \implies \begin{cases} x_1 = -\sqrt[4]{9x_2 - 2} \\ x_2 = \sqrt[3]{\frac{x_1^2 x_2^2 - 3x_1^3 + 8}{6}} \end{cases}$$

3. *Алгебраическая система:*

$$\begin{cases} x_1 + x_1^2 - 2x_2 x_3 - 0.1 = 0 \\ x_2 + x_2^2 + 3x_1 x_3 + 0.2 = 0 \\ x_3 + x_3^2 + 2x_1 x_2 - 0.3 = 0 \end{cases} \implies \begin{cases} x_1 = 0.1 - x_1^2 + 2x_2 x_3 \\ x_2 = -0.2 - x_2^2 - 3x_1 x_3 \\ x_3 = 0.3 - x_3^2 - 2x_1 x_2 \end{cases}$$

4. *Смешанная алгебраическая система:*

$$\begin{cases} x_1 - \sqrt{1 - x_2^2} = 0 \\ x_2 - x_1^3 = 0 \end{cases} \implies \begin{cases} x_1 = \sqrt[3]{x_2} \\ x_2 = \sqrt{1 - x_1^2} \end{cases}$$

5. Трансцендентная система:

$$\begin{cases} x_1 + e^{x_1} - 2 = 0 \\ x_2 + \sin(x_2) - x_1 = 0 \end{cases} \implies \begin{cases} x_1 = \ln(2 - x_1) \\ x_2 = x_1 - \sin(x_2) \end{cases}$$

Ниже приведены результаты решения указанных систем методом простых итераций с заданной точностью $\varepsilon = 10^{-5}$.

№	$\vec{x}^{(0)}$	$\vec{x}^*$	σ^2	Итерации
1	$(0.0, 0.0)^T$	$(-0.0, -0.0)^T$	0	1
2	$(-1.9, 2.1)^T$	$(-2.0, 2.0)^T$	3.76559^{-12}	22
3	$(0.0, 0.0, 0.0)^T$	$(-0.0179359, -0.249651, 0.235557)^T$	1.79298^{-15}	29
4	$(0.8, 0.6)^T$	$(0.826031, 0.563624)^T$	4.43422^{-15}	79
5	$(0.4, 0.2)^T$	$(0.442854, 0.222341)^T$	4.63827^{-15}	584

Таблица 1 — Сводная таблица результатов тестирования.

3 Заключение

После выполнения лабораторной работы у меня появились ответы на следующие вопросы.

Почему используемый критерий останова может быть неточным?

В работе мы допустили, что метод сходится быстро. Если же сходимость «медленная» ($q \rightarrow 1$), то разность двух соседних приближений может быть очень малой, в то время как само решение всё еще находится далеко от текущей точки. В таких ситуациях использование упрощенного критерия без учета q может привести к преждевременной остановке алгоритма до достижения требуемой точности.

Почему ряд тестов не получилось пройти на платформе Code&Test?

Полагаю, что причина кроется в высокой чувствительности к начальным условиям и виду эквивалентной системы. Если для тестовой системы спектральный радиус матрицы Якоби $q \geq 1$ в окрестности корня, то итерационный процесс расходится.

Какая асимптотическая сложность у алгоритма? Временная сложность метода простых итераций составляет $\mathcal{O}(M \cdot n)$, где M — количество затраченных итераций, а n — размерность системы. Как следует из блок-схемы, на каждой итерации происходит однократное вычисление вектора-функции $\Phi(x)$ и расчет нормы невязки за линейное время $\mathcal{O}(n)$. Здесь мы допускаем, что расчёт значения отображения Φ занимает константное время.

Пространственную сложность можно оценить $\mathcal{O}(n)$, поскольку алгоритм требует выделения памяти лишь под два вектора размерности n , что эффективнее прямых методов, требующих хранения квадратных матриц.

В чём преимущество метода простых итераций? Если сравнивать с методом Ньютона, главное преимущество — вычислительная простота одного шага. Метод простых итераций не требует вычисления элементов матрицы Якоби и решения системы линейных алгебраических уравнений на каждой итерации. Если сравнивать с методом градиентного спуска, то простому итерационному процессу не нужно подбирать какие-либо гиперпараметры (в частности, шаг обучения).

Какие недостатки обнаружены у метода простых итераций? Главный недостаток — линейная скорость сходимости, которая существенно уступает квадратичной сходимости метода Ньютона.

Кроме того, метод требует трудоемкого предварительного этапа: ручного преобразования исходной системы $F(x) = 0$ к эквивалентному виду $x = \Phi(x)$. При этом преобразование должно быть выполнено так, чтобы гарантировалось выполнение достаточного условия сходимости.

Конечно, есть эмпирические модификации метода простых итераций, которые позволяют формировать преобразование Φ автоматически. Однако их рассмотрение выходит за рамки лабораторной работы и лекционного материала.

Почему число итераций у трансцендентной системы резко возросло?

В разделе с условиями сходимости метода было сказано, что её скорость линейна и зависит от нормы матрицы Якоби. Увеличение количества итераций лишь говорит о том, что q оказалось очень близко к единице. То есть каждый последующий шаг лишь незначительно приближает нас к ответу, и алгоритму требуется большое число итераций для достижения требуемой точности.